

Mysterious Parody Bits

Lab 1 - COMP211 - Fall 2026

“Education never ends, Watson. It is a series of lessons, with the greatest for the last.”

~ Sherlock Holmes, *The Red Circle*

Part 1 of 3. A `xeh` spell reversed is `hex`

In this part of the lab you will write `hex.c` to convert plaintext ASCII to its hexadecimal representation. This program will be the opposite of the last one.

`hex.c` will read input as plaintext ASCII, until EOF is reached, and produce each character’s hexadecimal representation in ASCII characters 0-9 and A-F.

Here is an example usage:

```
$ echo "Hello" | ./hex
48656C6C6F0A
'H' -> 48
'e' -> 65
'l' -> 6C
'l' -> 6C
'o' -> 6F
'\n' -> 0A
```

Where is that newline coming from? `echo` prints a newline by default (unless `-n` is used).

```
$ echo -n "Hello" | ./hex
48656C6C6F
```

`hex.c` is the complement of `xeh.c`. Passing one’s output into the other’s input, via a pipe, should give you the first’s input:

```
$ echo "Hello" | ./hex | ./xeh
Hello
$ echo "48656C6C6F0A" | ./xeh | ./hex
48656C6C6F0A
```

You should always print a newline (`\n`) at the end of your generated hex output (as shown in the code below).

Pause for Questions

Before continuing, answer the Gradescope questions on piping.

Where to begin

In `hex.c`, you have been given a simple program that reads input one character at a time and prints the character right back out. You will build on top of this.

You already did the opposite work in `xeh.c`. This time, instead of combining two hex digits into one byte, you’re splitting one byte into two hex digits.

Here is an ascii table for your reference.

You’re ready to get to it implementing `hex.c`!

Testing

Start with the basic examples given earlier. Check that your outputs match.

```
$ echo "Hello" | ./hex
48656C6C6F0A

$ echo "Hello" | ./hex | ./xeh
Hello

$ echo "48656C6C6F0A" | ./xeh | ./hex
48656C6C6F0A
```

Writing Additional Test Cases

The program `hex.c` is the reverse of `xeh.c`:

- `xeh` converts hex to ASCII
- `hex` converts ASCII to hex

Because of this, we can reuse the same test cases you already created, just in the opposite direction.

Step 1 Create the test directories `~bash mkdir -p tests/hex/{input,expected,actual}~`

Step 2: Copy the existing tests in the opposite direction

The expected ASCII output from Part 0 becomes the input for Part 1:

```
cp tests/xeh/expected/*.expected tests/hex/input/
```

The hex input from Part 0 becomes the expected output for Part 1:

```
cp tests/xeh/input/*.hex tests/hex/expected/
```

Step 3: Rename the test files. Change to the input directory:

```
learncli$ cd tests/hex/input/
```

Rename the files that end in `.expected` so that they end in `.in`:

```
learncli$ for f in *.expected; do mv "$f" "${f%.expected}.in"; done
```

Then return to the root of your repository:

```
learncli$ cd ../../..
```

Check your work with `tree`. You should now have files such as:

```
tests/hex/
|-- input/
|   |-- digits.in
|   |-- hello.in
|   `-- ...
|-- expected/
|   |-- digits.hex
|   |-- hello.hex
|   `-- ...
`-- actual/
```

Step 4: Compile `hex`

From the root of your project directory:

```
learncli$ gcc hex.c -Wall -Wextra -std=c11 -g -o hex
```

Make sure there are no compiler errors.

Step 5: Test `hex` using the `.in` files

Example: All Digits

```
./hex < tests/hex/input/digits.in > tests/hex/actual/digits.actual  
diff tests/hex/actual/digits.actual tests/hex/expected/digits.hex
```

- No output from `diff` means the test passed
- Any output means the test failed

Step 6: Run all tests automatically

You can use a script similar to the one from Part 0.

Create a file named `hex_tests.sh`

Here's a guide for the changes you will need to make to the test script:

1. Set up the test directories. Your script needs to know where the input, expected, and actual files are stored. Look at how you defined these directories in `xeh_tests.sh`. What paths should you use for the `hex` tests?
2. Loop through the input files. In Part 0, your loop ran once for every input file in the `xeh` input directory. For `hex`, the input files end in `.in`. Modify the loop so that it processes every `.in` file in the `hex` input directory.
3. Find the test name. Each input filename contains the name of the test. For example:

```
tests/hex/input/digits.in -> name is digits  
tests/hex/input/upper.in  -> name is upper
```

In Part 0, you used `basename` to find this name. Modify that command for the suffix used by the Part 1 input files.

4. Construct the expected and actual filenames. Once you have the test name, use it to determine the paths of the corresponding expected and actual files. For a test named `digits`, ask yourself: Where should `digits.hex` be located? Where should `digits.actual` be created?
5. Run `hex`. Redirect the test's `.in` file into `./hex` and redirect the program's output into the corresponding `.actual` file.
6. Check the result. Use `diff` to compare what your program actually produced with what it was expected to produce. As in Part 0, print `<name>: PASS` when the files match and `<name>: FAIL` when they do not.

Once you have finished writing your test, make it executable:

```
$ chmod +x tests/hex/hex_tests.sh
```

Run the tests:

```
$ ./tests/hex/hex_tests.sh
```

Expected output:

```
digits: PASS  
upper: PASS  
lower: PASS  
punct: PASS  
spaces: PASS  
newlines: PASS
```

Pause for Questions

Before continuing, answer the Gradescope questions on `hex_tests.sh`.

Step 4: Clean up temporary files

```
$ rm tests/hex/*.tmp
```

Line Wrapping

If the hex output is 80 characters or longer, then it should “wrap” the output by inserting new line characters after every 80 hex digits.

```
$ echo "The world is full of obvious things which nobody by any chance ever observes." | ./hex
54686520776F726C642069732066756C6C206F66206F6276696F7573207468696E67732077686963
68206E6F626F647920627920616E79206368616E63652065766572206F627365727665732E0A
```

Education never ends

Once your `hex` program is working, hex encode the following string: `Education never ends` by running the command below.

```
$ echo "Education never ends" | ./hex
```

Then, use the generated hex string to navigate to the following URL, substituting the placeholder with your encoded string.

<http://bit.ly/your-encoding-here>